Verificación de software

Jesús Sánchez Cuadrado
Última actualización: 14/04/2026

Este tema se introduce el concepto de verificación de software y su importancia en el desarrollo de sistemas. Se presentarán dos técnicas de verificación en detalle: las pruebas de software y el análisis estático.

Índice

1. Introducción 1
2. Terminología 1
   2.1. Defectos, fallos y errores 1
   2.2. Verificación y validación 2
3. Pruebas de software 4
   3.1. Casos de prueba 4
   3.2. Principios de las pruebas de software 5
   3.3. Niveles de pruebas 6
   3.4. Regression testing (Pruebas de regresión) 8
4. Pruebas de caja negra 9
   4.1. Selección de datos de entrada 9
5. Pruebas de caja blanca 13
   5.1. Cobertura de código 14
6. Análisis estático 19
   6.1. Ejemplo de análisis estático: SonarQube 20
   6.2. Distinción entre análisis estático y testing 20
   6.3. Tipos de analizador estáticos 21
   6.4. Análisis arquitectural 24
Bibliografía 24

1. Introducción

Durante el proceso de desarrollo es inevitable introducir errores en el software que estamos construyendo. Es decir, debemos asumir que siempre que estemos programando vamos a cometer errores. Por tanto, la obligación de un desarrollador de software es aplicar técnicas para conseguir que el software esté libre de errores. Estas técnicas se engloban dentro del concepto de aseguramiento de la calidad del software (SQA, Software Quality Assurance), que incluyen actividades como la revisión de código, las pruebas de software, el análisis estático, la integración continua, etc.

En este tema se estudiarán estos conceptos, en particular centrándose en las pruebas de software y en el análisis estático.

2. Terminología

2.1. Defectos, fallos y errores

A la hora de hablar de errores o fallos en el software hay tres conceptos que es importante distinguir y que habitualmente utilizamos de manera intercambiable (definidos en ISO/IEC/IEEE 24765:2017 **Systems and software engineering – Vocabulary**).

- **Fallo (*failure*).** Es un **comportamiento** incorrecto del software. Es importante enfatizar la idea de que un fallo está relacionado con el comportamiento del sistema en tiempo de ejecución. Es decir, es un error que se manifiesta cuando el sistema está en funcionamiento.
- **Defecto o bug (*fault*).** Es la parte del código que es incorrecta. Un defecto o bug es una condición necesaria pero no suficiente para que haya un fallo. Por ejemplo, el siguiente código contiene un defecto porque se intenta dividir por cero, pero en realidad nunca se va a manifestar en tiempo de ejecución porque esa parte del código nunca se ejecuta.

  ```
  if (false) {
    y = 1 / 0;
    // Esta línea tiene un bug.
    // Se manifiesta como un fallo con una excepción
  }
  ```

  Es habitual utilizar ***bug*** para referirnos a aquellos defectos que se han manifestado en el software en producción, mientras que los defectos

que detectamos utilizando técnicas de desarrollo (ej., pruebas) son simplemente defectos.

- **Error.** Es la causa de un defecto. Por ejemplo, un error puede ser un error de diseño o un error de implementación. Es decir, qué es lo que ha fallado durante el desarrollo para que haya un defecto en el código. En el ejemplo anterior, el error es que se ha dividido por cero.

No todos los defectos provocan fallos en tiempo de ejecución. Por ejemplo, el siguiente código contiene un defecto[1][2], pero el código cliente no lo ejercitará en tiempo de ejecución porque de manera defensiva captura un error potencial.

```
/**
  * Devuelve la temperatura de una sala.
  * Se devuelve como un número codificado como una cadena.
  */
public String calcularTemperatura() {
  if (esGradosFarenheit()) {
    return "20F";
  } else {
    return "20";
  }
}

var temperatura = calcularTemperatura();
try {
  float temperaturaNumerica = Float.parseFloat(temperatura);
  mostrarTemperatura(String.format("%.2f", temperaturaNumerica));
} catch (NumberFormatException e) {
  mostrarTemperatura("--");
}
```

2.2. Verificación y validación

Es importante distinguir entre dos términos que a menudo se confunden: verificación y validación.

- **Verificación**: ¿se está construyendo el producto correctamente?
- **Validación**: ¿se está construyendo el producto correcto?

La diferencia entre estos dos conceptos es sutil pero importante:

- La verificación está orientada al desarrollo del software y al uso correcto de las técnicas de desarrollo.
- La validación está orientada a determinar si el software cumple los requisitos del cliente.

La verificación implica comprobar que las decisiones que se han tomado durante el desarrollo del software son correctas y no contienen errores (esto es, fallos o defectos). La validación implica comprobar que el software cumple con los requisitos.

[1] Es un ejemplo ilustrativo, la interfaz de la función es mala.
[2] El error es devolver `"20F"` que no puede ser convertido a *float* por la línea 14.

Por tanto, las técnicas de verificación y validación son complementarias y ambas son necesarias para asegurar la calidad del software. En este curso nos centraremos en las técnicas de verificación fundamentalmente.

2.2.1. Verificación

Hay diferentes técnicas para ***verificar*** software, entre las que se incluyen:

- **Pruebas (testing)**. Las pruebas de software son una técnica de ***verificación dinámica*** puesto que requieren ejecutar el software. El objetivo es planificar la ejecución del programa con ciertas entradas que provoquen la ocurrencia de un fallo, y a partir de ahí analizar qué ha ocurrido para determinar el ***bug*** correspondiente.
- **Análisis estático**. Es una técnica de verificación estática, lo que quiere decir que se realiza sin necesidad de ejecutar el programa, sino analizando el código fuente del mismo.
- **Inspecciones**. Las inspecciones de código consisten en revisar el código (y resto de artefactos de desarrollo) creados por otros desarrolladores para comprobar diferentes factores de calidad, como la corrección, la legibilidad, cumplimiento de los estándares de la empresa, etc.
- **Pruebas formales**. Dada una especificación formal que describe el comportamiento esperado del programa, una prueba formal garantiza que la implementación satisface la implementación. [1]

Cada una de estas técnicas tiene su utilidad dentro del proceso de desarrollo y también atendiendo al tipo de software que se esté construyendo.

En este curso nos centraremos en las pruebas de software y en el análisis estático.

2.2.2. Validación

Hay multitud de técnicas para ***validar*** software, entre las que se incluyen:

- Pruebas de aceptación. Evalúan el sistema desde la perspectiva del cliente o usuario final. Se basan en criterios de aceptación definidos a partir de requisitos funcionales y de negocio. Incluyen pruebas de aceptación de usuario (UAT), pruebas alfa y beta. Su objetivo es decidir si el sistema es aceptable para su uso o entrega.

- Pruebas de usabilidad. Analizan cómo los usuarios interactúan con el sistema para verificar que es fácil de usar, eficiente y comprensible. Se realizan con usuarios reales ejecutando tareas representativas. Permiten detectar problemas de experiencia de usuario que no se identifican con pruebas técnicas.

- Pruebas en entorno real. Consisten en evaluar el software en condiciones reales de operación (entorno productivo o similar). Incluyen pruebas piloto o despliegues controlados. Permiten validar que el sistema funciona correctamente en el contexto real de uso, considerando factores externos como usuarios, carga y entorno operativo.

Aunque estas técnicas no se verán en este curso, la idea clave es que se centran en el usuario y el contexto real en el que se ejecuta el software, no en la implementación interna del software.

Estático vs. dinámico

En el contexto de la programación, y de los lenguajes de programación, se dice que cierta característica es **estática** cuando su implementación tiene que ver con el análisis del código fuente. En cambio, se dice que algo es **dinámico** cuando tiene que ver con la ejecución.

Un ejemplo concreto es el tipado dinámico y el tipado estático. En el tipado dinámico la corrección de los tipos se comprueba en tiempo de ejecución. Por ejemplo, en Python la expresión `0 + '1'` provoca un error al ejecutar el programa. En cambio, en el tipado estático es el compilador el que analiza el programa y decide (sin ejecutarlo) qué expresiones son válidas. En el caso anterior emitiría un error de tipos.

En general, las características dinámicas son más flexibles mientras que las estáticas son más seguras (y más conservadoras).

[1] **Pruebas formales** A pesar de que ser una técnicas muy pesada, las pruebas formales han demostrado utilizado para detectar fallos en software muy probado. Por ejemplo, en [1] se describe cómo se detectó un error en el algoritmo de ordenación de Java a través de una prueba formal.

De hecho, Amazon hace uso extensivo de métodos formales para validar su infraestructura más crítica [2].

Como ejemplo de pruebas formales más o menos ligeras, el sistema Dafny es capaz de verificar formalmente invariantes y postcondiciones. En el siguiente ejemplo (ver https://dafny.org/latest/OnlineTutorial/guide) Dafny es capaz de informar de que la postcondición (expresada con `ensures`) no se cumple cuando `y` es un número negativo.

```
method MultipleReturns(x:
int, y: int) returns (more:
int, less: int)
  ensures less < x < more
{
  more := x + y;
  less := x - y;
}
```

3. Pruebas de software

Las pruebas de software son una técnica de **verificación dinámica**. Consisten en ejecutar el software con el objetivo de encontrar errores. Por tanto, para poder ejecutar una prueba necesitamos datos de entrada y para comprobar si el resultado obtenido es correcto necesitamos el resultado esperado o algún mecanismo que nos lo indique.

La Figura 1 muestra el esquema de las pruebas de software. Un programa tiene una serie de entradas posibles (el dominio de entrada). La ejecución del programa genera, para cada una de estas entradas, una salida dentro del dominio de salida. Por tanto, lo ideal sería poder comprobar que para cada entrada se obtiene la salida esperada. Sin embargo, en la práctica esto es imposible porque el dominio de entrada es potencialmente infinito.

En la práctica no podemos ejecutar todas las posibles entradas, así que se elegirán algunas entradas representativas, que serán un subconjunto del dominio de entrada. Por tanto, podemos decir que las pruebas de software son una *aproximación optimista*, en el sentido que la hipótesis es que si encontramos un dato de entrada para el que programe falla, igualmente fallará para datos similares.

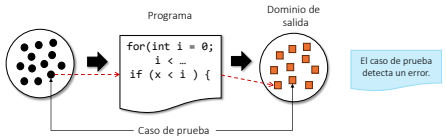

Figura 1: Esquema del proceso de testing.

3.1. Casos de prueba

Un test es exitoso si encuentra un defecto. *Goodenough and Gerhart, Towards a Theory of Test Data Selection*

Un **caso de prueba** es una especificación de valores utilizados como entrada para ejecutar una parte de un programa, junto con un mecanismo para comprobar si los valores de salida del programa (su comportamiento observable) es correcto.

En general, un caso de prueba tiene varias partes.

- Código necesario para configurar el software para que se puede ejecutar la parte de interés.
- Los datos de entrada.
- El resultado esperado o un predicado que verifique si el resultado es correcto o no (un oráculo).

Un **test suite** es un conjunto de casos de prueba.

El las pruebas de software el **oráculo** es el mecanismo que definimos para comprobar si la salida obtenida es correcta o no. El mecanismo básico es proporcionar un valor de salida esperado. Por ejemplo:

```
assert 10 == sumar(5, 5);
```

Sin embargo, este no es el único tipo de oráculo. Por ejemplo, para probar que un algoritmo optimizado es correcto se puede utilizar un programa anterior, más simple e ineficiente como oráculo. Incluso un humano realizado pruebas manuales actúa como oráculo.

3.2. Principios de las pruebas de software

1. **Las pruebas demuestran la presencia de defectos, no su ausencia**. Cada vez que ejecutamos un caso de prueba se está probando una pequeña parte del dominio de entrada. Así, si un caso de prueba encuentra un error, está demostrando la existencia de un defecto. Sin embargo, como no es posible ejecutar el programa para todo el dominio de entrada, entonces no es posible afirmar que si ninguna prueba encuentra errores estamos garantizando la ausencia de los errores (que podrían descubrirse quizás con un caso de prueba que no hemos creado). Por tanto, las pruebas reducen la probabilidad de que queden defectos ocultos en el software pero, incluso si no se encuentra ningún defecto, no constituye evidencia de corrección.

2. **Las pruebas exhaustivas no existen (o son imposibles)**. Probar todo con todas las combinaciones de entradas y precondiciones no es factible, excepto en casos triviales. Por ejemplo, para un formulario con 10 "checkboxes" (valores sí/no) implicaría probar 2^{10} combinaciones.

3. **Detectar errores pronto es más barato**. Es mucho más sencillo arreglar un defecto cuando se ha encontrado en la fase de desarrollo que si el sistema está en producción.

4. **Agrupación o *clustering* de defectos**. Los defectos suelen agruparse en ciertos módulos "problemáticos", por ejemplo porque sean más complejos de desarrollar. Por tanto, es útil dedicar más esfuerzo a crear pruebas en aquellos módulos donde se han encontrado más defectos Esto no quiere decir que se descuiden las áreas donde se han encontrado menor densidad de defectos sino que se pruebe en la proporción adecuada.

5. **Tener cuidado con la paradoja del pesticida**. Si repetimos las mismas pruebas una y otra vez, eventualmente estas pruebas ya no detectarán ningún defecto nuevo. Para detectar nuevos defectos, debemos actualizar las pruebas existentes y los datos de las pruebas y también escribir pruebas nuevas. (Las pruebas ya no son eficaces para encontrar defectos, al igual que los pesticidas después de un tiempo ya no son eficaces para matar insectos.)

6. **Las pruebas dependen del contexto**. Las pruebas se realizan de manera diferente dependiento del tipo de aplicación o sistema.Por ejemplo, si se está probando una aplicación empresarial las pruebas se enfocarán en la lógica de negocio, en la integración de sistemas, etc., mientras que si se está probando un compilador las pruebas se enfocarán en la corrección del código máquina generado, etc.

7. **Falacia de ausencia de errores** Este principio hace referencia a la creencia creencia errónea de que un sistema sin defectos conocidos será útil en producción. Sin embargo, que no se hayan detectado errores no garantiza que el sistema satisfaga las expectativas del

usuario o los requisitos comerciales. Para ello, son necesarias técnicas de validación del software.

3.3. Niveles de pruebas

Las pruebas de software se pueden realizar a diferentes niveles dependiendo del ámbito del código que se está probando. Habitualmente se consideran los siguientes niveles:

- Pruebas unitarias (*unit testing*). Se prueban las unidades de código de manera aislada. Por unidad de código se entiende un método o un clase. La principal restricción es que no tenga dependencias con componentes externos.
- Pruebas de Integración. El objetivo es garantizar que los módulos funcionen bien una vez que se integran. Para ello se combinan los módulos (por ejemplos, por pares) y se comprueban que las interfaces (y su implementación) son las esperadas.
- Pruebas de sistema. Sirven para probar el sistema al completo e incluyen también pruebas relativas a requisitos no funcionales, como rendimiendo, seguridad, usabilidad, etc.
- Pruebas ade Aceptación. Estas pruebas están relacionadas con la validación del software. Sirven para comprobar si el sistema satisface los requisitos desde el punto de vista del usuario.

La cantidad de pruebas y la frecuencia con la que se deben ejecutar se suele expresar con la llamada **pirámide de pruebas**.

Figura 2: Pirámide de pruebas.

Además de estas pruebas, durante el proceso de desarrollo es importante ejecutar las pruebas con frecuencia. Se denominan **pruebas de regresión** al hecho de ejecutar las pruebas cada vez que se modifica un programa con el objetivo de encontrar errores a través de las pruebas ya existentes.

3.3.1. Pruebas unitarias

El objetivo de las pruebas unitarias es probar una única característica del código, aislándolo de las dependencias que pueda tener con respecto a otros módulos del sistema.

Por ejemplo, si hemos escrito un método que calcula el precio con descuento de un producto para escribir un caso de prueba (al nivel de las pruebas unitarias) deberemos:

```
// 1. Crear un producto con los datos de entrada relevantes para
la prueba
Producto p = new Producto("prueba", 27.0);
// 2. Ejecutar el trozo de código de que se quiere probar (el
método descuento en este caso)
double descuento = producto.aplicarDescuento();
// 3. Comprobar que el resultado es correcto (utilizar un oráculo)
assertEquals(2.7, descuento);
```

Las características de las pruebas unitarias son las siguientes:

- Rápidas de ejecutar.
- Fáciles de escribir. Configurar el código para probar un método o una clase debería ser sencillo, en general.
- Fáciles de depurar. Si una prueba falla, sabemos que el error está en la unidad de código que estamos probando.

El principal problema de las pruebas unitarias es que no representan la realidad del sistema completo, si no solo partes aisladas. El hecho que todas las pruebas unitarias pasen no garantiza que el software funcione correctamente. Algunos errores solo se manifiestan cuando se ejecuta el sistema completo o cuando se integran diferentes componentes.

3.3.2. Pruebas de integración

Las pruebas de integración se centran en verificar las interacciones y el intercambio de datos entre los diferentes componentes o módulos de un sistema. Los componentes que se prueban pueden ser internos o externos al sistema. Los componentes internos son aquellos que forman parte del sistema que estamos desarrollando (por ejemplo, dos clases que tienen responsabilidades diferentes) y los componentes externos son aquellos que están fuera de los límites de nuestro sistema (por ejemplo, una base de datos, un servicio web, etc.).

Hay dos estrategias básicas para las pruebas de integración:

- Todos los módulos a la vez: "big bang integration testing". Si se ha utilizado una arquitectura como la arquitectura hexagonal entonces este modo de prueba de integración es relativamente sencillo.
- Incremental. Consiste en comprobar las interfaces entre los módulos por partes.

3.3.3. Pruebas de sistema

Las **pruebas de sistema** (o **end-to-end**) evalúan el sistema completo e integrado para garantizar que cumple con todos los requisitos funcionales y no funcionales. Se realizan en un entorno que simula las condiciones reales de producción, verificando el comportamiento de extremo a extremo (end-to-end).

La configuración, automatización y ejecución de este tipo de pruebas suele ser algo costoso ya que hay que crear un entorno que emule lo que vería el usuario final. Para esto se utilizan herramientas de ***testing*** especializadas como Playwright, Cypress o Selenium.

A continuación se muestra un ejemplo el que se usa Selenium para simular la interacción de un usuario con una web.

```java
public class ForumSystemTest {
  private WebDriver driver;

  @BeforeEach
  public void setup() {
    driver = new ChromeDriver();
    // URL donde está arrancado el servidor
    // Hay que arrancarlo de alguna forma
    driver.get("http://localhost:8080/forum");
  }

  @AfterEach
  public void cleanup()
    if (driver != null) {
      driver.quit();
```

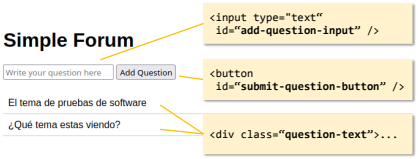

Figura 3: Página web anotada con el HTML de cada bloque. La prueba E2E utiliza los identificadores para simular la navegación y observar los resultados.

```
    }
  }

  @Test
  public void testPreguntas() {
    driver.findElement(By.id("add-question-input")).sendKeys("1 +
1 = ?");
    driver.findElement(By.id("submit-question-button")).click();
    // Verificar que las preguntas aparecen en la lista
    List<WebElement> questions =
driver.findElements(By.className("question-text"));
    assertFalse(questions.isEmpty(), "No se encontraron preguntas
en el foro.");
    assertEquals("1 + 1 = ?", questions.get(0).getText());
  }
}
```

3.4. Regression testing (Pruebas de regresión)

Las **pruebas de regresión** son una estrategia de pruebas utilizada para volver a ejecutar pruebas existentes cada vez que se añade nueva funcionalidad o se arregla un defecto.

Se denomina **regresión** a un tipo particular de defecto que ocurre tras producir un cambio en el código.

Una buena discusión puede encontrarse aquí: https://github.com/resources/articles/regression-testing-definition-types--and-tools

La idea fundamental es que tras realizar un cambio en el software es posible que alguno de los módulos que funcionaban falle. Esto puede ocurrir por múltiples motivos, por ejemplo, porque el cambio afecte a la configuración de la aplicación y "rompa" otros módulos, por un mal diseño de la aplicación que hace cambios en un módulo afecten a otros, debido a dependencias implícitas entre los módulos, asunciones incorrectas al introducir el nuevo cambio, etc.

Por tanto, es necesario volver a ejecutar el *test suite* del proyecto para asegurar que el nuevo cambio *al menos* no ha hecho que algo que se había comprobado que funcionaba[2], haya dejado de funcionar.

[2]Como se ha explicado, con las pruebas no podemos garantizar que "algo funcione".

3.4.1. Tipos de pruebas

Habitualmente se distingue entre dos tipos de pruebas según si el desarrollador "mira" o tiene acceso al código fuente o no(independientemente del nivel de testing al que se apliquen).

- **Pruebas de caja negra** (**testing funcional** o **pruebas basadas en la especificación**). En este tipo de pruebas el desarrollador parte de la especificación del software, esto es, requisitos funcionales, especificación del comportamiento de un algoritmo, diseño del software, etc.). La prueba se escribe para comprobar si la implementación cumple con tal especificación. Por tanto, en este tipo de pruebas el desarrollador **no conoce la implementación**.
- **Pruebas de blanca** (**testing estructural**). En este tipo de pruebas el objetivo es escribir pruebas que prueben las diferentes estructuras

del código (líneas de código, condicionales, bucles, etc.). Es decir, el desarrollador **sí conoce la implementación**.

4. Pruebas de caja negra

Las **pruebas de caja negra** se centran en el comportamiento del sistema, sin tener en cuenta su implementación interna.

Las principales caracteristicas de las pruebas de caja negra son:

- Se centran en el dominio del problema, no en la implementación. Por ejemplo, se pueden basar en los requisitos.
- No es necesario conocer el código por lo que se pueden realizar antes de que el código esté implementado.
- Ayudan a capturar errores lógicos.
- Se pueden aplicar a cualquier nivel de pruebas.

Puesto que las pruebas de caja negra se centran en el dominio del problema, nos ayudan a asegurar que estamos cubriendo los comportamientos importantes del sistema. Sin embargo, no garantizan que se estén cubriendo todos los caminos posibles del sistema. Para poder aplicar pruebas de caja negra es necesario tener una especificación del sistema. Por tanto, con estas pruebas estamos ejercitando el sistema con respecto a la lógica de esta especificación y así es posible que descubramos defectos lógicos en el sistema[3].

El hecho de que no haga falta la implementación nos permite realizar las pruebas antes de que el código esté implementado, y por ejemplo puede facilitar prácticas como *Test-Driven Development*.

[3] Por defecto lógico nos referimos un error en la implementación debido a que no está alineada con la especificacion, bien por una mala compresión de éstos o por un bug

4.1. Selección de datos de entrada

Un aspecto fundamental de las pruebas de caja negra es cómo elegir los valores de entrada.

4.1.1. Aproximación simple

Una forma simple de seleccionar los valores de entrada para una prueba es probar con todas las posibles entradas, es decir, *testing exhaustivo*.

El problema es que para cualquier sistema de tamaño real, el número de casos de prueba posibles es prácticamente infinito. Incluso si cada ejecución de un caso de prueba es muy rápida, no tenemos tiempo suficiente para ejecutarlos todos.

Por tanto, es necesario seleccionar un subconjunto de casos de prueba. Para ello hay varias estrategias.

4.1.2. Random testing

Las generación de pruebas aleatoria es una estrategia de pruebas que se basa en seleccionar del conjunto de entradas posible un número n de valores de forma aleatoria.

La principal ventaja de las pruebas aleatorias es que son simples de implementar (no hay pensar realmente en qué valores de entrada elegir) y que no hay sesgo del *tester* en el sentido de que las entradas no las elege la persona

que está probando y por tanto no se corre el riesgo de elegir entradas que no hacen fallar el programa.

Sin embargo, hay dos problemas fundamentales:

1. Los defectos no se distribuyen uniformemente, sino que (como veremos) suelen depender de la especificación.
2. No es sencillo diseñar el oráculo cuando las pruebas son aleatorias porque las entradas

no están seleccionadas manualmente. [4]

[4] ¿Qué tipo de oráculos encajan bien con las pruebas aleatorias?

4.1.3. Partition testing

Se desea elegir manualmente entradas que sean especialmente relevantes para poder descubrir errores de la implementación con respecto a la especificación. La observación clave para conseguir esto es que, en la práctica, **los errores similares se acumulan en la misma zona**.

Las **pruebas basadas en particiones** (o *partition testing*) consiste en identificar conjuntos de entradas para los cuales el programa se debe comportar igual independientemente de los valores concretos que se elijan para las pruebas.

Para conseguir este objetivo se define el concepto de **clase de equivalencia**.

Una **clase de equivalencia** (*equivalence partition*) es un conjunto de valores para los que se puede asumir, de manera razonable, que la ejecución de la prueba provocará salidas similares.

Esto permite reducir el espacio de entrada de las pruebas ya que, en teoría, es suficiente probar solo algunos valores de cada clase de equivalencia.

Ejemplo: Supongamos una función que clasifica a un solicitante de un préstamo en tres categorías dependiendo de sus ingresos mensuales.

```
/**
 * Este servicio determina la categoría de un solicitante dependiendo
 * de sus ingresos mensuales, atendiendo a las siguiente reglas:
 * - Para ingesos menores de 1000 euros al mes, el solicitante es no elegible.
 * - Para ingresos igual o superiores a 1000 euros, puede acceder al préstamos básico
 * - Para ingesos superiores a 3000 euros, puede acceder al préstamo premium
 */
public class ServicioPrestamo {
  public Elegibilidad calcularElegibilidad(double ingresos) { ... }
}
```

En este caso hay tres particiones de equivalencia:

- $0 < \text{ingresos} < 1000€$
- $1000 \leq \text{ingresos} \leq 3000€$
- $\text{ingresos} \geq 1000$

⚠ **Importante**

No es necesario disponer de la implementación para poder elegir las particiones de entrada porque las pruebas de caja negra se basan en la especificación.

Clases válidas e inválidas. Además, podemos clasificar las particiones en clases válidas e inválidas, dependiendo de cómo el sistema debe comportarse ante ellas. Así tenemos:

- Las clases **válidas** son aquellas para la que la entradas del caso de prueba deben ser procesadas de manera correcta por el sistema (esto es, satisfacen las pre-condiciones del sistema).
- Las clases **inválidas** son aquellas que el sistema debe rechazar (en lugar de fallar de manera no recuperable), por ejemplo lanzando una excepción o devolviendo un valor especial.

En el ejemplo anterior, las clases válidas son los ingresos mayores o iguales a cero mientras que las clases inválidas son los ingresos menores que cero.

¿Cómo sería un caso de prueba para `ServicioPrestamo`? A continuación se muestra una implementación en JUnit 5 donde se instancia cada partición con un caso de prueba concreto.[5]

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class ServicioPrestamoTest {
    private final ServicioPrestamo servicio = new
ServicioPrestamo();

    @Test
    void Usuario_NoElegible() {
        // Partición: 0 <= ingresos < 1000
        double ingresos = 500.0;

        Elegibilidad resultado =
servicio.calcularElegibilidad(ingresos);

        assertEquals(Elegibilidad.NO_ELEGIBLE, resultado);
    }

    @Test
    void Usuario_PrestamoBasico() {
        // Partición: 1000 <= ingresos <= 3000
        double ingresos = 2000.0;

        Elegibilidad resultado =
servicio.calcularElegibilidad(ingresos);

        assertEquals(Elegibilidad.BASICO, resultado);
    }

    @Test
    void Usuario_PestamoPremium() {
        // Partición: ingresos > 3000
        double ingresos = 4000.0;

        Elegibilidad resultado =
servicio.calcularElegibilidad(ingresos);
```

[5] Observa que `calcularElegibilidad` toma una tipo *double* para representar el dinero de los ingresos. Si utilizaramos un value object `Dinero` la clase inválida no habría que comprobarla en este caso de prueba, sino únicamente al probar el value object `Dinero`.

```
        assertEquals(Elegibilidad.PREMIUM, resultado);
    }

    /* Clase inválida */
    @Test
    void Usuario_IngresosNegativos() {
        // Partición inválida: ingresos < 0
        double ingresos = -100.0;

        assertThrows(IllegalArgumentException.class,
                () -> servicio.calcularElegibilidad(ingresos));
    }
}
```

4.1.4. Análisis de casos límite

Las técnicas de identificación de particiones permiten encontrar rangos de valores que dividen la entrada en partes diferenciadas. Sin embargo, para poder crear casos de prueba concretos es necesario disponer de los valores concretos con los que se desea ejecutar el programa. Es decir, hay que elegir valores interesantes como representantes de las clases de equivalencia. En el ejemplo anterior se ha elegido un valor cualquiera, ¿pero son estos los mejores valores que se pueden elegir?

La técnica de **análisis de casos límite** consiste en elegir como valores representativos de las clases de equivalencia los inmediatamente superiores, inferiores y los que están en el límite de las clases de equivalencia.

La razón detrás de esta técnica que es los fallos tienden a acumularse en los casos límite.

There are 2 hard problems in computer science: cache invalidation, naming things, and off-by-1 errors. https://martinfowler.com/bliki/TwoHardThings.html

En el ejemplo anterior, los puntos críticos son:

- 0€ (límite inferior válido)
- 1000€ (cambio de NO_ELEGIBLE a BASICO)
- 3000€ (cambio de BASICO a PREMIUM)

Por tanto, estaremos interesados en crear casos de prueba que utilicen los valores inmediatamente superiores e inferiores, y también que prueben los límites exactos. Aplicando la técnica de análisis de casos límite tendríamos:

- −0.01 → inválido (chequear que lanza excepción)
- 0 → NO_ELEGIBLE
- 0.01 → NO_ELEGIBLE
- 999.99 → NO_ELEGIBLE
- 1000 → BASICO
- 1000.01 → BASICO
- 2999.99 → BASICO
- 3000 → BASICO
- 3000.01 → PREMIUM

5. Pruebas de caja blanca

En las pruebas de caja negra la entrada para derivar los casos de prueba es la especificación del software y por tanto se está probando que el software es correcto con respecto a la especificación. Sin embargo, para poder encontrar un ***bug*** es necesario que haya al menos un caso de prueba que ejecute el trozo de código donde está el defecto (es decir, provocar el fallo del programa) y es posible que las pruebas de caja negra dejen parte del código sin probar. Algunas razones para esto son:

- Las pruebas de caja negra son subjetivas puesto que dependen de la interpretación de la especificación por el ***tester*** (que no necesariamente es el desarrollador).
- El desarrollador puede cometer un error y malinterpretar la especificación, y la selección de los valores de entrada no ejercita la parte del código donde está el error.
- La especificación es incompleta o ambigua. Quizás el desarrollador añade comportamiento que no está previsto en la especificación (para completarla) mientras que el ***tester*** no sabe que debe probarlo.
- En la práctica, hay decisiones en la implementación que no son visibles en la especificación. Por ejemplo, optimizaciones, manejo de errores, etc.

Las pruebas de caja blanca se basan en esta idea: crear casos de prueba "mirando" el código fuente para asegurar que las partes "interesantes" del código fuente son ejecutadas por los casos de prueba.

Las **pruebas de caja blanca** (*white box testing*) son aquellas que se realizan utilizando el conocimiento del código fuente.

Este tipo de pruebas asumen que el desarrollador tiene acceso al código fuente, y el objetivo es crear pruebas que ejerciten las diferentes estructuras del código.

Caja negra vs. caja blanca En las pruebas de caja negra el objetivo es que las pruebas cubran la especificación.

En las pruebas de caja blanca el objetivo es que las pruebas cubran el código fuente.

Las pruebas de caja blanca se basan en utilizar criterios de **cobertura de código** para disponer de una métrica que indique qué proporción del código ha sido ejecutada por las pruebas. Por ejemplo, la siguiente función no está bien especificada y además contiene un error en la implementación.

Ejemplo simple para pruebas de caja blanca.

```
1 /**
2  * La función devuelve el resultado de dividir a entre b.
3  */
4 double dividir(double a, double b) {
5   if (b == 0.0) {
6     return 0.0;  // <- Aquí hay un error
7   }
8   return a / b;
9 }
```

Al realizar pruebas de caja blanca con un único caso de prueba (ej., `dividir(10, 5) = 2`) el error no se detectaría. Utilizando pruebas de caja

blanca y aplicando un criterio simple de cobertura como "¿por qué líneas ha pasado la prueba?" podríamos observar que la línea 6 no ha sido probada. En este punto no sabemos si hay un defecto (porque no hemos escrito ninguna prueba), pero nos da una pista para escribir un caso de prueba que ejercite esa línea de código. Al hacerlo, descubríamos el defecto. [6]

[6] ¿Cómo tendría que ser el caso de prueba para demostrar el defecto?

5.1. Cobertura de código

La **cobertura de código** es una métrica que mide qué porcentaje del código ha sido ejecutado por conjunto de pruebas. Hay diferentes tipos de métricas de cobertura dependiendo de qué características del código se miden.

Herramientas de cobertura Para poder realizar pruebas de caja blanca de manera práctica es necesario disponer de herramientas de cobertura que analicen qué partes del código se han ejecutado.

Por ejemplo, para Java se suele utilizar https://www.eclemma.org/jacoco/, que se puede configurar en Maven y tiene un plug-in para Eclipse.

Existen diferentes métricas de cobertura, a saber:

- Cobertura de líneas
- Cobertura de condiciones
- Cobertura de ramas
- Cobertura de ramas + condiciones
- Cobertura de múltiples condiciones
- Cobertura MC/DC

5.1.1. Cobertura de sentencias o líneas

Es el tipo de cobertura más simple. Consiste en contar por qué líneas ha pasado la ejecución de los casos de prueba y compararla con el total de líneas del programa. Asumimos que cada línea de código tiene una sentencia (o al revés, que cada sentencia se escribe en una única línea).

$$\text{cobertura de lineas} = \frac{\text{sentencias ejecutadas}}{\text{número total de sentencias}} \cdot 100 \quad (1)$$

Como ejemplo concreto, supongamos que queremos probar el siguiente método utilizando pruebas de caja blanca y el criterio de cobertura de sentencias.

```
1 public double calcularPrecioFinal(double precio, boolean
  esMiembro) {
2     if (precio <= 0) {
3         return 0;
4     }
5
6     double descuento;
7
8     if (esMiembro) {
9         descuento = 0.20;
10    } else {
11        descuento = 0.05;
12    }
13
14    if (precio > 500) {
15        descuento += 0.05;
16    }
17
18    return precio * (1 - descuento);
```

```
19 }
```

Si utilizamos los dos casos de prueba siguientes, la cobertura que se obtiene es: $\frac{5}{9} \cdot 100 = 55.5\%$ con el primer caso de prueba y $\frac{7}{9} \cdot 100 = 77\%$ con el segundo caso de prueba (ver el listado que se muestra a continuación). Para calcular el total no se pueden sumar ambas coberturas (porque una incluye a la otra) sino que hay que contar el total de líneas ejecutadas que es $\frac{8}{9} \cdot 100 = 88\%$.[7]

[7] ¿Qué caso de prueba habría que crear para obtener un 100% de cobertura de líneas?

:

```
@Test
public void testMiembroSinExtra() {
  assertEquals(80, ejemplo.calcularPrecioFinal(100, true));
}

@Test
public void testMiembroConExtra() {
  assertEquals(540, ejemplo.calcularPrecioFinal(600, false));
}
```

```
1 public double calcularPrecioFinal(double precio, boolean
  esMiembro) {
2     if (precio <= 0) {
3         return 0;
4     }
5
6     double descuento;
7
8     if (esMiembro) {
9         descuento = 0.20;
10    } else {
11        descuento = 0.05;
12    }
13
14    if (precio > 500) {
15        descuento += 0.05;
16    }
17
18    return precio * (1 - descuento);
19 }
```

La cobertura por sentencias es muy usada en la práctica porque es fácil de interpretar. Un objetivo típico puede ser alcanzar entre un 80- 90% de cobertura Sin embargo presenta dos problemas principales:

- En primer lugar, la cobertura de sentencias cuenta igual una línea compleja que una simple.
- Además, ignora el hecho de no ejecutar un condicional que no tiene un *else*.

5.1.2. Cobertura de ramas

La **cobertura de ramas** (*branch coverage*) mide el porcentaje de ramas que han sido ejecutadas. A diferencia de la cobertura de líneas (que solo cuenta si una línea se ejecutó), la cobertura de ramas asegura que cada punto de decisión, como *if*, *else*, *switch* o condiciones de los *bucles*, se evalúen tanto como a verdadero como a falso.

$$\text{cobertura de ramas} = \frac{\text{ramas ejecutadas}}{\text{número total de ramas}} \cdot 100 \qquad (2)$$

El objetivo de la cobertura de ramas es ejecutar todas las ramas del programa, o dicho de otra forma, para obtener el 100% de cobertura de ramas hay que escribir casos de prueba que ejecuten todas las ramas. Para ello, esencialmente hay que elegir valores de entrada para los que las condiciones sean true y false.

El análisis de la cobertura de ramas es conveniente hacerlo utilizando el grafo de control de flujo (control flow graph; CFG) como base. La razón es que en el código las ramas correspondientes a los ***else*** están implícitas y no se "ven". La Figura 4 muestra el grafo de control de flujo para el ejemplo.

Siguiendo con el ejemplo anterior, para estos dos casos de prueba:

```
@Test
public void testPrecioInvalido_NoMiembro() {
  assertEquals(0, ejemplo.calcularPrecioFinal(-1, false));
}

@Test
public void testExtra_Miembro() {
  assertEquals(540, ejemplo.calcularPrecioFinal(600, true));
}
```

Se obtiene la cobertura de ramas que se muestra a continuación, que es: $\frac{3}{6} \cdot 100 = 50\%$

```
1 public double calcularPrecioFinal(double precio, boolean
  esMiembro) {
2     if (precio <= 0) {
3         return 0;
4     }
5
6     double descuento;
7
8     if (esMiembro) {
9         descuento = 0.20;
10    } else {
11        descuento = 0.05;
12    }
13
14    if (precio > 500) {
15        descuento += 0.05;
16    }
17
18    return precio * (1 - descuento);
19 }
```

En verde se marcan las condiciones donde se toman ambas ramas. En amarillo cuando sólo se ha tomado una rama. Para completar, en azul las líneas que se ejecutan y en rojo las que no se ejecutan.

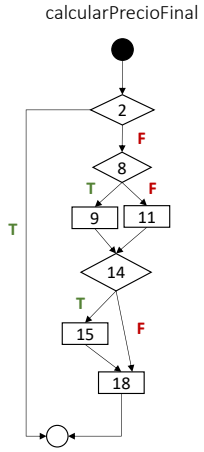

Figura 4: CFG del ejemplo. Los nodos están anotados con el número de línea. Los rombos representan decisiones.

Problemas de la cobertura de ramas. La cobertura de ramas funciona bien cuando no hay condiciones compuestas. El problema es que el comporta-

miento de una rama puede depender de los valores de las variables usadas en las condiciones. Por ejemplo, supongamos una variación del ejemplo anterior y de los casos de prueba que se muestran a continuación. La cobertura de ramas es del 100%, en cambio, el error no se detecta.

```
public double calcularPrecioFinal(double precio, boolean
esMiembro) {
  double descuento;

  // ERROR: debería ser && en lugar de ||
  // (porque el descuento máximo requiere ser miembro)
  if (precio > 500 || esMiembro) {
      descuento = 0.20;
  } else {
      descuento = 0.05;
  }

  return precio * (1 - descuento);
}

// Casos de prueba
calcularPrecioFinal(600, false);  // precio > 500 = true,
esMiembro = false
calcularPrecioFinal(100, false);  // precio > 500 = false,
esMiembro = false
```

5.1.3. Cobertura de condiciones

En este tipo de cobertura se pretende asegurar que cada condición individual del programa evalúe a ***true*** y ***false*** al menos una vez.

En el ejemplo anterior se podría conseguir un 100% de cobertura de condiciones si con los dos siguientes casos de prueba:

```
    ejemplo.calcularPrecioFinal(600, false);
    ejemplo.calcularPrecioFinal(100, true);
```

El primer caso de prueba evalúa `precio > 500 -> true` y `esMiembro -> false` mientras que el segundo evalúa `precio > 500 -> false` y `esMiembro -> true`.

El problema, sin embargo, es que la cobertura de ramas obtenida ahora es del 50%. De ahí, que en lugar de utilizar esta cobertura sóla se utilice en combinación con la cobertura de ramas y condiciones.

5.1.4. Cobertura de ramas y condiciones

La **cobertura de ramas y condiciones** tiene como objetivo cada decisión se ejecute al menos una vez con resultado ***true*** y una vez con resultado ***false***, y además, que cada condición individual se evalúe a ***true*** y a ***false*** al menos una vez.

$$\text{cobertura} = \frac{\text{ramas ejecutadas} + \text{condiciones cubiertas}}{\text{n}^{\text{o}} \text{ total ramas} + \text{n}^{\text{o}} \text{ total condiciones}} \cdot 100 \quad (3)$$

Con los casos de prueba anteriores, la cobertura de ramas y condiciones es: $\frac{2+1}{2+2} \cdot 100 = 75\%$.

En el ejemplo anterior, los casos de prueba con los que se consigue un 100% de cobertura de ramas y condiciones serían similares a los siguientes:

```
calcularPrecioFinal(600, false);  // precio > 500 = true,
esMiembro = false
calcularPrecioFinal(100, false);  // precio > 500 = false,
esMiembro = false

// Hay que probar ahora que esMiembro = true
calcularPrecioFinal(600, true);
```

Por tanto, la cobertura de ramas y condiciones garantiza que se recorren todas las ramas del grafo de flujo y además que cada condición individual se ha ejercitado tanto con ***true*** como ***false***.

Sin embargo, no se han probado todas las combinaciones posibles de condiciones (ej., en el ejemplo anterior se ha probado cuando `esMiembro = true y precio > 500` pero no cuando `esMiembro = true y precio <= 500`).

5.1.5. Cobertura de condiciones múltiples

Un criterio más fuerte sería comprobar todas las posibles combinaciones de valores para las condiciones. En el caso anterior, esto significaría probar $2^2 = 4$ combinaciones:

```
precio > 500 = true,   esMiembro = false
precio > 500 = false,  esMiembro = false
precio > 500 = true,   esMiembro = true
precio > 500 = false,  esMiembro = true
```

Sin embargo, a medida que el número de condiciones crece, esta aproximación no es manejable. Con 3 condiciones sería $2^3 = 8$, mientras que con 10 condiciones, sería $2^{10} = 1024$ condiciones.

Por tanto, este criterio no suele usarse. En su lugar se utiliza MC/DC.

5.1.6. Cobertura MC/DC (Modified Condition/Decision Coverage)

El criterio **MC/DC** tiene como objetivo utilizar aquellas condiciones individuales cuyos valores afecten al resultado global de la condición. Lo que se pretende es encontrar un compromiso entre la comprobación exhaustiva (criterio de condiciones múltiples) y el esfuerzo a la hora de realizar las pruebas. El MC/DC es mucho "más barato" que probar todas las combinaciones puesto que se puede conseguir una cobertura del 100% con $N + 1$ casos de prueba, donde N es el número de condiciones individuales.

El criterio MC/DC es el estándar en sistemas críticos. En industrias como la aviación (norma DO-178C), la automoción (ISO 26262) o el ferrocarril, el MC/DC es obligatorio por ley.

Cálculo del MC/DC:. Supongamos la siguiente expresión: `A || B`. La tabla de verdad es:

	A	B	Resultado
1	True	True	True
2	True	False	True
3	False	True	True

4	False	False	False

Los casos que se consideran es:

- Casos 2 y 4. B tiene el mismo valor, pero cuando A cambia el resultado cambia.
- Casos 3 y 4. A tiene el mismo valor, pero cuando B cambia el resultado cambia.

Por tanto, son necesarios 3 casos de prueba, que serían: 2, 3 y 4.

Ejemplo concreto: Por ejemplo, supongamos el siguiente condicional que podría formar parte del controlador de un ***airbag***:

```
if (presion_alta && sensor_activado && !sistema_bloqueado &&
velocidad > 20) {
  // desplegar el airbag
}
```

El MC/DC asegura que has probado que cada una de esas variables, por sí sola, es capaz de cambiar el resultado final. No basta con que el airbag se despliegue; hay que demostrar que si el `sistema_bloqueado` es verdadero, el airbag no se despliega, independientemente de lo demás.

5.1.7. Subsumpción

Se dice que un criterio A **subsume** a otro criterio B si todo conjunto de pruebas que cumple A automáticamente cumple B.

Dicho de otra forma, si se obtiene el 100% de cobertura en A, también se obtiene el 100% en B, pero no al revés. Esta idea sirve para razonar sobre qué criterios proporcionan garantías más fuertes de cobertura. La Figura 5 muestra la relación de subsumpción entre los diferentes criterios. Los dos criterios más fuertes son ***cobertura de condiciones múltiples*** y ***MC/DC***, pero también es más costoso obtener los casos de prueba.

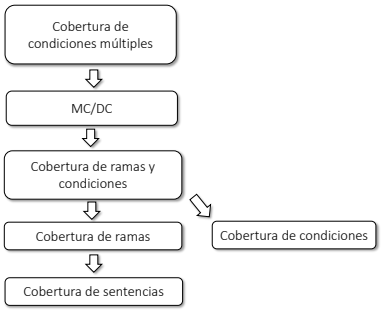

Figura 5: Relación de subsumpción entre criterios de cobertura.

Dependiendo del tipo de aplicación se decidirá utilizar un criterio u otro. Por ejemplo, para aplicaciones empresariales utilizar cobertura de líneas o de ramas puede ser suficiente. Para sistemas críticos (automoción, aviónica, etc.) la cobertura tiene que ser al menos MC/DC.

6. Análisis estático

El **análisis estático** hace referencia a las técnicas que sirven para razonar sobre el comportamiento del programa sin ejecutarlo. Así, un **analizador estático** es un programa que lee otro programa con el objetivo de analizarlo. Cada analizador estático tiene un propósito diferente, como puede ser, detectar errores, detectar vulnerabilidades o errores de seguridad, cumplimiento de los estándares de programación, detectar problemas de rendimiento, etc.

Los compiladores de los lenguajes tipados estáticamente ya realizan (al menos) un tipo concreto de análisis estático: la comprobación de tipos. También pueden realizar otro tipo de comprobaciones, como por ejemplo los compiladores de C/C++ que permiten activar ***warnings*** más avanzados con `-Wall`. En todo caso, lo habitual es complementar el análisis básico proporcionado por los compiladores con analizadores estáticos centrados en la detección de otro tipo de problemas.

6.1. Ejemplo de análisis estático: SonarQube

El siguiente ejemplo muestra un análisis estático realizado con SonarQube. En este caso, el analizador ha detectado que el recurso `BufferedReader` no se cierra, lo cual puede provocar una fuga de recursos.

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class Ejemplo {
    public static void main(String[] args) {
        try {
            BufferedReader br = new BufferedReader(new
FileReader("archivo.txt"));
            String linea = br.readLine();
            System.out.println(linea);
            // Problema: No se ha cerrado el recurso
            // SonarQube: "Resources should be closed" (regla
S2095)
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Algunas herramientas de análisis estático:

- SonarQube: https://www.sonarqube.org/
- FindBugs: https://findbugs.sourceforge.net/
- PMD: https://pmd.github.io/
- Checkstyle: https://checkstyle.sourceforge.io/
- ESLint: https://eslint.org/
- Pylint: https://pylint.pycqa.org/en/latest/

La forma correcta de escribir este código sería utilizando un bloque *try-with--resources* para asegurar que el recurso se cierra automáticamente, y en este caso SonarQube no indicaría ningún problema.

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class Ejemplo {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(new
FileReader("archivo.txt"))) {
            String linea = br.readLine();
            System.out.println(linea);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

6.2. Distinción entre análisis estático y testing

La principal diferencia entre el análisis estático y el testing es que el primero no ejecuta el programa mientras que el segundo sí lo hace. Por tanto, un analizador estático no necesita datos de entrada para funcionar (la entrada es el propio código fuente) mientras que el testing sí necesita datos de entrada (casos de prueba).

Un analizador estático, de cierta forma, considera todas las posibles entradas de manera implícita de manera que cuando detecta un problema se refiere a que el problema se puede producir para alguna entrada (aunque no da información de para qué entrada concreta se produce el problema). En cambio, el testing se centra en casos de prueba concretos y por tanto,

cuando detecta un problema, se refiere a que el programa falla para esa entrada concreta.

En el ejemplo anterior, el analizador estático es capaz de detectar el problema sin necesidad de un oráculo, mientras que detectar este problema con pruebas es mucho más difícil porque aunque en este ejemplo cualquier ejecución causaría el error, éste no se manifiesta en tiempo de ejecución hasta que el programa se ejecuta durante un tiempo suficiente como para que se produzca la fuga de recursos (lo cual no es algo que se pueda garantizar con un caso de prueba concreto).

Un problema del análisis estático es que tiene tendencia a generar falsos positivos. Un **falso positivo** es un error que el analizador estático detecta pero que en realidad no es un error porque no se puede construir una entrada que genere el fallo. Por ejemplo, en el siguiente código, SonarQube marcará `usuario.getNombre()` como posible NPE (Null Pointer Exception) porque `usuario` puede ser nulo. Sin embargo, si en ningún sitio del programa se invoca a `obtenerNombre(null)`, entonces el error que SonarQube detecta no es un error real, sino un falso positivo.

```
public class UsuarioService {

    public String obtenerNombre(Usuario usuario) {
      if (usuario == null) {
        System.out.println("Usuario nulo");
      }

      // SonarQube puede marcar esto como posible NPE
      return usuario.getNombre().toUpperCase();
    }
}
```

Al contrario, un **falso negativo** es un error que el analizador estático no detecta pero que en realidad sí es un error. Los analizadores estáticos no son perfectos y pueden no detectar ciertos errores por diversas razones: a) errores en la implementación del analizador, b) limitaciones en la capacidad de análisis, c) el propio analizador elige no detectar ciertos errores para evitar falsos positivos, etc.

En el caso del testing, no hay falsos positivos porque si el programa falla para una entrada concreta, entonces es un error real. Sin embargo, sí puede haber falsos negativos porque el programa puede contener errores que no se manifiestan con los casos de prueba que se han escrito.

Cada analizador estático se especializa en detectar un tipo de problemas concreto, por lo que es posible que un analizador no detecte un error que otro sí detecta. Por ejemplo, el analizador de SonarQube se centra en detectar problemas de seguridad, de calidad del código, etc.,

6.3. Tipos de analizador estáticos

Hay muchos tipos de analizadores estáticos dependiendo del tipo de problemas que detectan. Algunos tipos comunes son:

- Linters (estilo): Estilo y convenciones de código.

- Detectores de errores basados en patrones (bugs comunes): Problemas simples basadas en reglas.
- Procesadores de anotaciones (más información): Extiende el sistema de tipos.
- Análisis de flujo de datos (interpretación abstracta y más profunda): Análisis profundo del comportamiento del programa.

6.3.1. Linters

Un linter es un tipo de analizador estático que se centra en detectar problemas relacionados con el estilo de código, las convenciones de programación, la legibilidad, etc. El objetivo no es detectar errores funcionales, sino mejorar la calidad del código y facilitar su mantenimiento.

Un linter suele ser más rápido que otros tipos de analizadores estáticos porque se basa en reglas simples y patrones de código, ya que realiza un análisis ligero a nivel de la sintaxis del código fuente.

Por ejemplo, el linter de Java Checkstyle detectaría los siguientes problemas en el siguiente código:

```
import pds.*; // Error: import con comodines

// Error: clase pública sin mayúscula inicial
public class ejemplo {
    int var1 = 1; // Error: declaración sin separar por un líneaa
en blanco

    public void metodoEjemplo() {
       if(var1 > 3); // Error: Sentencia vacía
    }
}
```

Los linters suelen permitir configurar las reglas para adaptarlas a las convenciones de cada equipo o proyecto. Por ejemplo, https://github.com/checkstyle/checkstyle/blob/master/src/main/resources/google_checks.xml es un conjunto de reglas de estilo basado en las guías de estilo de Google para Java.

6.3.2. Analizadores basados en patrones

Un analizador basado en patrones es un tipo de analizador estático que se centra en detectar errores comunes o problemas de seguridad basados en patrones de código específicos. Estos analizadores tienen una base de datos de reglas o patrones que representan errores conocidos, y el analizador escanea el código fuente para encontrar coincidencias con esos patrones.

Un ejemplo típico de patrón que detectaría este tipo de analizador es el siguiente:

```
String a = new String("hola");
String b = new String("hola");

if (a == b) {  // Comparación incorrecta
    System.out.println("Son iguales");
}
```

El patrón que implementaría internamente este tipo de analizador sería algo como lo siguiente:

```
if (expr instanceof BinaryExpression) {
  if (expr.operator == "==" &&
      tipo(expr.left) == String &&
```

```
      tipo(expr.right) == String) {
    report("Comparación de strings con '==', se debería usar
'equals()'");
  }
}
```

En ocasiones los analizadores basados en patrones se extienden con análisis basados en el flujo de control y/o el flujo datos para detectar errores más complejos. Por ejemplo, podrían detectar un posible NPE (Null Pointer Exception) aunque no haya una comparación explícita con `null` si detectan que una variable puede ser nula y luego se accede a un método o propiedad de esa variable sin comprobar si es nula. En el siguiente ejemplo el analizador debe analizar el flujo de datos para detectar que hay un camino del programa en el que `usuario` pasa a ser nulo.

```
Usuario usuario = new Usuario();
if (i == 100) {
  usuario = null;
}

// Posible NPE cuando i == 100
System.out.println(usuario.getNombre());
```

6.3.3. Analizadores basados en anotaciones

Un analizador estático se basa en la información disponible en el código de fuente. Sin embargo, en muchas ocasiones hay errores que no se pueden detectar de manera precisa sin información adicional. Por ejemplo, para detectar un posible NPE (Null Pointer Exception) es necesario saber si una variable puede ser nula o no. En Java, una variable de un tipo no primitivo siempre puede ser nula por lo que el analizador siempre tiene que "ponerse en el peor" escenario y en un código como el siguiente, el analizador marcaría `usuario.getNombre()` como posible NPE. Esto es problemático porque posiblemente se generarán muchos falsos positivos, lo cual puede hacer que el analizador no sea útil.

```
public class UsuarioService {
    public String obtenerNombre(Usuario usuario) {
      return usuario.getNombre().toUpperCase();
    }
}
```

La solución adoptada por algunos lenguajes de programación es la de utilizar anotaciones para proporcionar información adicional al analizador estático. Por ejemplo, en Java se pueden utilizar las anotaciones `@Nullable` y `@NonNull` para indicar si una variable puede ser nula o no. De esta forma, el analizador puede detectar posibles NPE de manera más precisa y reducir los falsos positivos.

Por ejemplo, en el siguiente código al utilizar la anotación `@Nonnull` el analizador estático puede saber que `usuario.getNombre()` no puede ser nulo y por tanto no marcaría esa línea como posible NPE. De esta manera, se reduce el número de falsos positivos y el analizador es más útil para detectar errores reales.

En Java, las anotaciones como `@Nullable` y `@NonNull` no forman parte del lenguaje, sino que son proporcionadas por bibliotecas externas como `javax.annotation` o `org.jetbrains.annotations`. El proyecto https://jspecify.dev/ tiene como objetivo estandarizar estas anotaciones para mejorar la interoperabilidad entre diferentes herramientas.

```java
import javax.annotation.Nonnull;

public class UsuarioService {
    public String obtenerNombre(@Nonnull Usuario usuario) {
      return usuario.getNombre().toUpperCase();
    }
}
```

En la práctica, las anotaciones ***extienden el sistema de tipos*** del lenguaje de programación, para que el programador dote a los tipos de información adicional. De esta manera, el analizador estático actúa como un "compilador" que comprueba que el programa es correcto con respecto a ese sistema de tipos extendido. En el caso de las anotaciones `@Nullable` y `@NonNull`, el sistema de tipos extendido permite distinguir entre tipos que pueden ser nulos y tipos que no pueden ser nulos, lo cual permite al analizador detectar posibles NPE de manera más precisa.

6.4. Análisis arquitectural

Un tipo particular de análisis estático es el **análisis arquitectural**. En este tipo de análisis no se pretende detectar problemas funcionales, sino asegurar que el código fuente cumple con las reglas arquitecturales definidas para el proyecto.

Por ejemplo, en un proyecto que sigue una arquitectura hexagonal hay que asegurar las siguientes reglas:

- Las clases del dominio no pueden depender de las clases de la infraestructura (o adaptadores).
- Las clases de la capa de aplicación pueden acceder a las clases del dominio (esto es, los servicios dan acceso al dominio).
- Los adaptadores pueden acceder a las clases definidas en la capa de aplicación (esto es, los servicios de la aplicación).

Bibliografía

[1] S. De Gouw, F. S. De Boer, R. Bubel, R. Hähnle, J. Rot, y D. Steinhöfel, «Verifying OpenJDK's sort method for generic collections», ***Journal of Automated Reasoning***, vol. 62, n.º 1, pp. 93-126, 2019.

[2] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, y M. Deardeuff, «How Amazon web services uses formal methods», ***Communications of the ACM***, vol. 58, n.º 4, pp. 66-73, 2015.